LAPORAN PRAKTIKUM PBO 7

EXCEPTION HANDLING

Disusun Oleh:

Ananda Firdaus (251511003)

Kelas 2A

Jurusan Teknik Komputer dan Informatika

Kamis, 22 Oktober 2026

Program Studi D-3 Teknik Informatika

Politeknik Negeri Bandung

2026

Repo: https://github.com/FirdausSinau/PBO

BAGIAN A - CountLetters

A.1 Code awal

Program membaca satu kata, mengubahnya menjadi huruf besar, lalu menghitung frekuensi tiap huruf. Index array diperoleh dari `word.charAt(i) - 'A'`, tanpa pengecekan apakah karakter tersebut benar-benar huruf.

```
import java.util.Scanner;

public class CountLetters {
    public static void main(String[] args) {
        int[] counts = new int[26];
        Scanner scan = new Scanner(System.in);

        System.out.print("Enter a single word (letters only, please): ");
        String word = scan.nextLine();
        word = word.toUpperCase();

        for (int i = 0; i < word.length(); i++)
            counts[word.charAt(i) - 'A']++;

        System.out.println();
        for (int i = 0; i < counts.length; i++)
            if (counts[i] != 0)
                System.out.println((char)(i + 'A') + ": " + counts[i]);
    }
}
```

Input satu kata berjalan normal.

```
Enter a single word (letters only, please): JAVA

A: 2
J: 1
V: 1
```

Huruf A muncul dua kali, J dan V masing-masing satu kali.

A.2 Bukti ArrayIndexOutOfBoundsException

Input berikutnya adalah frasa yang mengandung spasi dan tanda baca.

```
Enter a single word (letters only, please): JAVA PBO!

Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index -33 out of bounds for
length 26
    at CountLetters.main(CountLetters.java:13)
```

Spasi memiliki kode karakter 32. Perhitungan `32 - 65` menghasilkan `-33`, dan index negatif tidak sah untuk array berukuran 26.

Exception terjadi pada loop pertama, yaitu baris 13. Karena tidak ada handler, eksekusi `main` berhenti di titik itu. Loop pencetakan huruf tidak pernah dijalankan, sehingga tidak ada output frekuensi sama sekali.

A.3 Code try-catch

Body loop pertama dibungkus `try`, dengan `catch` kosong. Handler kosong dipakai untuk melihat efek penangkapan exception tanpa menambah output lain.

```
for (int i = 0; i < word.length(); i++) {
    try {
        counts[word.charAt(i) - 'A']++;
    } catch (ArrayIndexOutOfBoundsException e) {
    }
}
```

Output untuk input `JAVA PBO!`:

```
Enter a single word (letters only, please):
A: 2
B: 1
J: 1
O: 1
P: 1
V: 1
```

Program tidak berhenti lagi. Karakter spasi dan `!` dilewati, sedangkan semua huruf tetap dihitung.

A.4 Output setelah handler diperbaiki

Handler kosong menyembunyikan informasi tentang karakter yang dilewati. Handler kemudian diubah agar menampilkan karakter penyebab exception.

```
import java.util.Scanner;

public class CountLetters {
    public static void main(String[] args) {
        int[] counts = new int[26];
        Scanner scan = new Scanner(System.in);
```

```
        System.out.print("Enter a single word (letters only, please): ");
        String word = scan.nextLine();
        word = word.toUpperCase();

        for (int i = 0; i < word.length(); i++) {
            try {
                counts[word.charAt(i) - 'A']++;
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("Not a letter: " + word.charAt(i));
            }
        }

        System.out.println();
        for (int i = 0; i < counts.length; i++)
            if (counts[i] != 0)
                System.out.println((char)(i + 'A') + ": " + counts[i]);
    }
}
```

Output:

```
Enter a single word (letters only, please): Not a letter:
Not a letter: !

A: 2
B: 1
J: 1
O: 1
P: 1
V: 1
```

Baris pertama berisi dua spasi setelah titik dua. Satu spasi berasal dari teks `"Not a letter: "`, satu lagi adalah karakter spasi yang sedang dilaporkan.

A.5 Mengapa loop tetap berjalan

Posisi `try-catch` berada di dalam badan `for`. Saat spasi diproses, exception dilempar, lalu eksekusi berpindah ke `catch` pada iterasi yang sama. Setelah `catch` selesai, `for` melanjutkan iterasi berikutnya.

Handler tidak memperbaiki karakter yang salah. Handler hanya memutuskan bahwa karakter itu dilewati, lalu proses lanjut. Keputusan itulah yang mencegah program berhenti.

BAGIAN B - ParseInts

B.1 Code awal

Program memecah satu baris teks menjadi token, mengubah setiap token menjadi integer, lalu menjumlahkannya.

```
import java.util.Scanner;

public class ParseInts {
    public static void main(String[] args) {
        int val, sum = 0;
        Scanner scan = new Scanner(System.in);
```

```
        System.out.println("Enter a line of text");
        Scanner scanLine = new Scanner(scan.nextLine());

        while (scanLine.hasNext()) {
            val = Integer.parseInt(scanLine.next());
            sum += val;
        }

        System.out.println("The sum of the integers on this line is " + sum);
    }
}
```

Input yang seluruhnya angka berjalan benar.

```
Enter a line of text
10 20 30 40
The sum of the integers on this line is 100
```

Input yang mengandung kata gagal saat token pertama diproses.

```
Enter a line of text
We have 2 dogs and 1 cat.

Exception in thread "main" java.lang.NumberFormatException: For input string: "We"
    at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
    at java.base/java.lang.Integer.parseInt(Integer.java:527)
    at java.base/java.lang.Integer.parseInt(Integer.java:624)
    at ParseInts.main(ParseInts.java:12)
```

`Integer.parseInt()` melempar `NumberFormatException` karena `"We"` bukan representasi integer. Exception naik melalui pemanggilnya sampai keluar dari `main`, lalu program berhenti.

B.2 try-catch di luar while

Seluruh loop dibungkus satu `try`, dan `catch` diletakkan setelah loop.

```
try {
    while (scanLine.hasNext()) {
        val = Integer.parseInt(scanLine.next());
        sum += val;
    }
} catch (NumberFormatException e) {
}
```

Output:

Input	Sum
`We have 2 dogs and 1 cat.`	0
`1 fish 2 fish`	1

Pada input pertama, token `"We"` langsung gagal sehingga tidak ada angka yang sempat dijumlahkan. Pada input kedua, angka `1` sempat dijumlahkan, lalu `"fish"` gagal dan proses berhenti. Angka `2` setelahnya tidak pernah dibaca.

B.3 try-catch di dalam while

`try-catch` dipindahkan ke dalam badan while, sehingga setiap token memiliki kesempatan diproses sendiri.

```
while (scanLine.hasNext()) {
    try {
```

```
            val = Integer.parseInt(scanLine.next());
            sum += val;
        } catch (NumberFormatException e) {
        }
    }
```

Output:

Input	Sum
`We have 2 dogs and 1 cat.`	3
`1 fish 2 fish`	3
`10 20 30 40`	100

B.4 Perbedaan alur

Penempatan	Cakupan try	Saat token invalid	Token berikutnya
Di luar while	seluruh loop	keluar dari try, yang berarti keluar dari while	tidak diproses
Di dalam while	satu iterasi	berpindah ke catch pada iterasi itu	diproses

Perbedaannya bukan pada tipe exception atau isi handler, melainkan pada luas blok `try`. Blok yang mencakup seluruh loop membuat satu kegagalan menghentikan keseluruhan proses. Blok yang hanya mencakup satu iterasi membuat kegagalan terbatas pada token tersebut.

BAGIAN C - Factorials

C.1 Hasil baseline yang tidak valid

`MathUtils.factorial()` menghitung faktorial dengan loop menurun. Tidak ada pengecekan domain.

```
public class MathUtils {
    public static int factorial(int n) {
        int fac = 1;
        for (int i = n; i > 0; i--)
            fac *= i;
        return fac;
    }
}
```

`Factorials.main()` meminta input berulang selama pengguna menjawab `y`.

```
import java.util.Scanner;

public class Factorials {
    public static void main(String[] args) {
        String keepGoing = "y";
        Scanner scan = new Scanner(System.in);

        while (keepGoing.equals("y") || keepGoing.equals("Y")) {
            System.out.print("Enter an integer: ");
            int val = scan.nextInt();
            System.out.println("Factorial(" + val + ") = "
                    + MathUtils.factorial(val));
```

```
            System.out.print("Another factorial? (y/n) ");
            keepGoing = scan.next();
        }
    }
}
```

Output baseline:

```
Enter an integer: 5
Factorial(5) = 120
Another factorial? (y/n) y
Enter an integer: 0
Factorial(0) = 1
Another factorial? (y/n) y
Enter an integer: -1
Factorial(-1) = 1
Another factorial? (y/n) y
Enter an integer: 17
Factorial(17) = -288522240
Another factorial? (y/n) n
```

Dua hasil tidak valid.

`factorial(-1)` mengembalikan `1` karena loop dimulai dari `i = -1` dan kondisi `i > 0` langsung bernilai false. Loop tidak pernah dijalankan, sehingga `fac` tetap bernilai awal `1`.

`factorial(17)` menghasilkan bilangan negatif karena `17!` bernilai 355.687.428.096.000, jauh di atas batas `int` (2.147.483.647). Java memotong hasilnya menjadi 32 bit tanpa memberi error.

Nilai `1` untuk input negatif berbahaya karena sama dengan hasil yang sah untuk `factorial(0)` dan `factorial(1)`. Pemanggil tidak bisa membedakan hasil benar dari kegagalan, sehingga angka salah dapat dipakai seolah-olah valid.

C.2 Code throw IllegalArgumentException

Method menolak nilai negatif sebelum perhitungan dimulai.

```
public static int factorial(int n) throws IllegalArgumentException {
    if (n < 0) {
        throw new IllegalArgumentException(
            "factorial tidak didefinisikan untuk bilangan negatif: " + n);
    }

    int fac = 1;
    for (int i = n; i > 0; i--)
        fac *= i;
    return fac;
}
```

C.3 Bukti exception sebelum catch

`Factorials` belum memiliki handler. Input `-1` menghasilkan:

```
Enter an integer: -1
Exception in thread "main" java.lang.IllegalArgumentException: factorial tidak didefinisikan untuk
bilangan negatif: -1
    at MathUtils.factorial(MathUtils.java:4)
    at Factorials.main(Factorials.java:12)
```

Stack trace menunjukkan arah perjalanan exception. Exception dibuat di `MathUtils.factorial` baris 4, naik ke `Factorials.main` baris 12, dan karena tidak ada `catch` yang cocok, program berhenti.

Di tahap ini `MathUtils` yang mendeteksi kondisi invalid. `Factorials` belum menangani exception tersebut.

C.4 Code handler

Handler diletakkan di dalam loop, membungkus hanya bagian perhitungan.

```
while (keepGoing.equals("y") || keepGoing.equals("Y")) {
    System.out.print("Enter an integer: ");
    int val = scan.nextInt();

    try {
        System.out.println("Factorial(" + val + ") = "
                + MathUtils.factorial(val));
    } catch (IllegalArgumentException e) {
        System.out.println("Tidak dapat menghitung: " + e.getMessage());
    }

    System.out.print("Another factorial? (y/n) ");
    keepGoing = scan.next();
}
```

`scan.nextInt()` sengaja diletakkan di luar `try` agar handler hanya menangani exception dari perhitungan faktorial.

C.5 Output input valid, negatif, dan lebih dari 16

`MathUtils` kemudian ditambah pengecekan batas atas dengan message yang berbeda.

```
public class MathUtils {
    public static int factorial(int n) throws IllegalArgumentException {
        if (n < 0) {
            throw new IllegalArgumentException(
                "nilai tidak boleh negatif: " + n);
        }
        if (n > 16) {
            throw new IllegalArgumentException(
                "nilai terlalu besar untuk factorial berbasis int: " + n);
        }

        int fac = 1;
        for (int i = n; i > 0; i--)
            fac *= i;
        return fac;
    }
}
```

Output pengujian batas:

```
Enter an integer: 5
Factorial(5) = 120
Another factorial? (y/n) y
Enter an integer: 0
Factorial(0) = 1
Another factorial? (y/n) y
Enter an integer: -1
Tidak dapat menghitung: nilai tidak boleh negatif: -1
Another factorial? (y/n) y
Enter an integer: 16
Factorial(16) = 2004189184
Another factorial? (y/n) y
Enter an integer: 17
Tidak dapat menghitung: nilai terlalu besar untuk factorial berbasis int: 17
Another factorial? (y/n) n
```

Input	Hasil	Keterangan
5	120	benar
0	1	benar
-1	message negatif, loop lanjut	ditolak
16	2004189184	diterima sesuai batas modul
17	message terlalu besar, loop lanjut	ditolak

Semua input invalid ditangani tanpa mengakhiri program.

C.6 Catatan batas overflow

Modul menetapkan batas `n > 16` untuk latihan ini, dan batas tersebut dipakai sesuai instruksi. Hasil pengujian tambahan menunjukkan overflow sebenarnya sudah terjadi lebih awal:

```
Enter an integer: 12
Factorial(12) = 479001600
Another factorial? (y/n) y
Enter an integer: 13
Factorial(13) = 1932053504
Another factorial? (y/n) n
```

`12!` bernilai 479.001.600 dan masih muat di `int`. `13!` bernilai 6.227.020.800, melewati batas `int`, sehingga hasil yang tercetak salah walaupun bertanda positif. Batas yang lebih tepat untuk `int` adalah 12, bukan 16.

Pengecekan `n > 16` tetap memenuhi skenario modul karena menolak 17 ke atas. Catatan ini menunjukkan bahwa batas numerik harus dihitung dari kapasitas tipe data, bukan hanya dari contoh input.

BAGIAN D - Tabel Skenario Uji

D.1 Hasil T7-01 sampai T7-08

ID	Skenario	Expected Result	Hasil	Status	Bukti
T7-01	CountLetters dengan satu kata	Frekuensi huruf tercetak tanpa exception	A:2 J:1 V:1	PASS	A.1
T7-02	CountLetters dengan spasi/tanda baca sebelum catch	ArrayIndexOutOfBoundsException terlihat	Index -33 out of bounds for length 26	PASS	A.2
T7-03	CountLetters setelah catch	Karakter non-huruf dilewati dan proses lanjut	Not a letter: spasi dan !, semua huruf tercetak	PASS	A.4
T7-04	ParseInts try-catch di luar while	Proses berhenti pada token invalid pertama	Sum 0 dan sum 1	PASS	B.2
T7-05	ParseInts try-catch di dalam while	Semua token diproses, integer tetap dijumlahkan	Sum 3, 3, dan 100	PASS	B.3

T7-06	factorial(-1) setelah throw, sebelum catch	IllegalArgumentException menghentikan alur	Exception dan program berhenti	PASS	C.3
T7-07	factorial(-1) setelah catch di loop	Message tampil dan program dapat lanjut	Message negatif, lanjut ke 5 dan menghasilkan 120	PASS	C.5
T7-08	factorial(17)	Exception dengan message batas lebih dari 16 ditangani	Message terlalu besar, loop lanjut	PASS	C.5

D.2 Ketiga exception termasuk unchecked

`ArrayIndexOutOfBoundsException`, `NumberFormatException`, dan `IllegalArgumentException` adalah turunan `RuntimeException`. Compiler tidak memaksa ketiganya untuk ditangkap atau dideklarasikan. Exception baru terlihat saat program berjalan, dan program berhenti hanya jika tidak ada handler yang cocok.

Karena itu, `throws IllegalArgumentException` pada `MathUtils.factorial` tidak diwajibkan oleh compiler. Deklarasi tersebut tetap ditulis agar pembaca method tahu bahwa input di luar domain akan ditolak.

BAGIAN E - Refleksi

1. Mengapa CountLetters berhenti saat menemukan karakter non-huruf

Program awal tidak memeriksa apakah karakter berada di antara `A` dan `Z`. Karakter spasi menghasilkan index `-33`, dan akses `counts[-33]` melempar `ArrayIndexOutOfBoundsException`. Tidak ada handler, sehingga `main` berhenti sebelum loop pencetakan dijalankan.

Setelah `catch` ditambahkan di dalam loop, karakter yang gagal diproses dilewati. Loop lanjut ke karakter berikutnya, dan huruf setelah spasi tetap dihitung. Output berubah dari tidak ada sama sekali menjadi daftar frekuensi yang lengkap.

2. Apakah exception selalu berarti aplikasi harus berhenti

Tidak. Pada ParseInts dengan `try-catch` di dalam while, token `"We"` melempar `NumberFormatException`, tetapi program tetap memproses `2`, `1`, dan token lainnya. Hasil akhirnya sum 3, bukan berhenti.

Pada Factorials, input `-1` juga ditolak dengan message, lalu loop tetap berjalan dan input `5` dihitung menjadi 120. Exception menghentikan operasi yang gagal, bukan seluruh aplikasi, asalkan ada handler di posisi yang tepat.

3. Mengapa posisi try-catch mengubah hasil ParseInts

Saat `try` mencakup seluruh while, exception menyebabkan eksekusi keluar dari blok `try`. Karena while berada di dalam blok itu, loop ikut ditinggalkan. Token setelah token invalid tidak pernah dibaca.

Saat `try` hanya mencakup satu iterasi, exception menyebabkan eksekusi keluar dari iterasi tersebut menuju `catch`. While berada di luar blok `try`, sehingga loop tetap hidup dan iterasi berikutnya berjalan.

4. Mengapa throw lebih aman daripada mengembalikan nilai palsu

`factorial(-1)` pada baseline mengembalikan `1`, nilai yang sama dengan `factorial(0)` dan `factorial(1)`. Pemanggil tidak punya cara membedakan hasil benar dari kegagalan. Nilai tersebut bisa disimpan atau dipakai dalam perhitungan lain sebagai angka yang salah.

Dengan `throw`, method tidak mengembalikan nilai apa pun untuk input invalid. Pemanggil dipaksa menghadapi kegagalan tersebut, baik dengan `catch` maupun dengan membiarkannya naik. Tidak ada nilai palsu yang bisa lolos diam-diam.

5. Perbedaan peran throw dan catch

`throw` adalah cara method melaporkan bahwa keadaannya tidak valid dan operasi normal tidak bisa diselesaikan. `throw` berada di `MathUtils.factorial` dan menghentikan eksekusi method tersebut.

`catch` adalah keputusan pemanggil tentang apa yang harus dilakukan setelah laporan itu diterima. `catch` berada di `Factorials.main` dan memilih menampilkan message lalu melanjutkan loop. `throw` memberi sinyal, `catch` menentukan kebijakan pemulihan.

6. Manfaat message berbeda pada IllegalArgumentException

Handler menangkap berdasarkan tipe exception, bukan berdasarkan penyebab. Kedua kondisi melempar `IllegalArgumentException`, sehingga keduanya masuk ke `catch` yang sama. Message adalah satu-satunya pembeda yang sampai ke pengguna.

Dengan message `"nilai tidak boleh negatif"` dan `"nilai terlalu besar untuk factorial berbasis int"`, pengguna langsung tahu input mana yang salah dan mengapa. Tanpa message yang spesifik, pengguna hanya tahu bahwa perhitungan gagal, tanpa petunjuk untuk memperbaikinya.

7. Pertanyaan tentang checked exception

Ketiga exception pada praktikum ini tidak dipaksa oleh compiler. Pertanyaan yang muncul ketika membandingkan dengan checked exception adalah: apa yang menentukan sebuah exception wajib ditangani saat compile, mengapa operasi file termasuk checked, dan apakah posisi handler juga berubah ketika exception wajib dideklarasikan.

Pertanyaan lain: jika checked exception memaksa `try-catch` atau `throws`, apakah itu membuat program lebih aman, atau hanya memindahkan kewajiban menangani ke method pemanggil.

8. Kalimat penutup

Exception handling bukan sekadar mencegah program crash, tetapi memisahkan deteksi masalah dari keputusan untuk melanjutkan, sehingga kesalahan tidak berubah menjadi nilai palsu dan program dapat pulih di titik yang tepat.

KESIMPULAN

Tiga kasus menunjukkan tiga peran exception yang berbeda. CountLetters memakai `catch` untuk melewati karakter yang tidak valid. ParseInts menunjukkan bahwa posisi `try-catch` menentukan cakupan recovery. Factorials menunjukkan bahwa method dapat menolak input dengan `throw`, lalu pemanggil memutuskan untuk melanjutkan.

Ketiga program final berhasil dikompilasi dan seluruh skenario T7-01 sampai T7-08 berstatus PASS.